Attestor

A cross-chain loan repayment history you cannot lie to — built on Attestcoin.

BUIDL CTC 2026 FALL · ETHEREUM SEPOLIA → CREDITCOIN CC3 TESTNET · GITHUB.COM/RICHARDREKI/ATTESTOR

THE PROBLEM

Creditcoin exists to make credit history verifiable. But the money that builds that history — stablecoin loans and repayments — lives on Ethereum. Bridging the *record* across chains usually means trusting a relayer's word that a repayment happened. A credit history is worthless if it can be forged.

THE SOLUTION

A borrower repays on Ethereum; an autonomous agent posts it to a credit history on Creditcoin — but the record is written **only** if an Attestcoin proof shows the repayment really happened. The agent proposes; the on-chain book disposes. **Neither the agent nor we can write a false entry.**

ETHEREUM SEPOLIA

Borrower calls `repay()` — real USDC moves to the lender, emits `Repaid`.

OFF-CHAIN AGENT

Watches, applies a risk gate, waits ~8 min for attestation, fetches the proof, proposes a posting.

CREDITCOIN CC3

`AttestedLoanBook` verifies the proof on the BlockProver precompile, re-derives it, writes the history.

Remove Attestcoin and there is no product: the book holds no trusted input of its own. Every write to a credit history is downstream of one precompile call — `0x…0FD2 . verify(…)` returning without reverting.

THE SECURITY MODEL — ATTESTCOIN PROVES *LESS* THAN IT APPEARS

It proves a transaction was **included in a confirmed block** — not that it *succeeded*, called the right contract, or was sent by who it claims. The book enforces, on-chain, every property the proof does not:

CHECK	WITHOUT IT
`status == 1`	a reverted repayment posts as if it paid
`chainId (resolved)`	a valid proof from the wrong chain is accepted
`source contract`	any contract's call is posted as a repayment
`repay selector`	a different function passes as a repayment
`borrower == sender`	credit is assigned to whoever relays the proof
`freshness + maxAge`	an attested repayment is postable forever
`tx-hash consumed`	one repayment is posted again and again

Plus economic guards: zero-amount and self-payment repayments are refused on both chains, because an entry is only worth something if real money moved. **28 contract tests; 22 reject a forgery.**

WHAT WE FOUND THAT THE DOCS DON'T SAY — SEVEN UNDOCUMENTED PROTOCOL FACTS

1. **The precompile reverts on a bad proof; it does not return** `false` as the SDK documents — so `if (!verified)` is dead code.
2. **~8-minute latency is a real constraint:** a 32-block reorg window named only in an error string, plus ~39 blocks of attestation lag. We surface it, not hide it.
3. `chainKey` **is environment-scoped** — key 1 is Sepolia on testnet, Ethereum mainnet on mainnet. Bind to `chainId`, or a promotion silently changes trust.
4. **SDK and chain names differ:** the function is `verify` on-chain, not the SDK's `verifySingle`; ChainInfo is snake_case on-chain.
5. **The proof envelope is** `(uint8,bytes[])`**, not** `bytes[]` — and the SDK's own docs say the wrong one. A consumer written from the docs decodes nothing.
6. **The precompiles can't be fork-tested, and the failure is silent** — a call to the codeless address succeeds and returns nothing, passing the wrong assertion.
7. `get_chain_by_key` **returns one tuple, not two values** — a bug we shipped; every mocked test passed, it reverted every time on-chain. It's why we built a live-check layer.

Findings 5 and 7 make a documentation-following consumer fail outright. They are our ecosystem contribution as much as the product is.

LIVE & VERIFIABLE — NO KEY, NO GAS

reproduce	`node spike.mjs`	MockUSD	0xCFd5E8e6…d0ef (Sepolia)
tests	`forge test` — 28 pass	LoanRepay	0x08F8b91A…9021 (Sepolia)
live	`node tools/live-check.mjs`	real repay	0x00a39800… (250 mUSD moved)
agent	20 tests, tsc clean	BlockProver	0x…0FD2 ChainInfo 0x…0FD3

HONEST STATUS & ROADMAP

> Security core complete; proof pipeline verified end-to-end against the live precompiles. Source half deployed on Sepolia with a real repayment proven. **The loan book on CC3 is written and tested but not yet deployed — the first on-chain posting is waiting on testnet funds.** The demonstrated repayment is the exact transaction it will post; nothing is mocked.

Next: deploy the book and run the first live posting · multi-loan credit scoring read by a Creditcoin lending contract · batch proofs (the SDK supports up to 10) · mainnet with real USDC and a governance-set source binding.